Project Two: Summary and Reflections Report

Aidan Leddy

Southern New Hampshire University

CS-320: Software Test, Automation QA

Prof. Sherif Antoun

October 18, 2026

Project Two: Summary and Reflections Report

In creating the Contact, Task, and Appointment services for the Grand Strand Systems mobile application, I analyzed various approaches and applied appropriate testing strategies, leveraging JUnit 5 tests to enforce customer requirements. My testing approach across all three features was closely aligned to the software requirements, as every functional constraint was mirrored with both positive tests (valid inputs) and negative tests (invalid inputs). Across all three domain models, the customer required unique, immutable IDs up to 10 characters that could not be null. In *ContactTest.java*, *TaskTest.java*, and *AppointmentTest.java*, I verified these constraints using tests like *testContactIdNull* and *testTaskIdTooLong*, and wrote reflection-based tests (*testContactIdImmutability* in *ContactTest.java* lines 78-87) to confirm that no ID setter exists and that each ID field is declared final. For feature-specific rules, *ContactTest.java* verified non-null first and last names (10 characters max), an address (30 characters max), and a 10-digit phone number through *testContactCreationAndGettersSuccess*, while targeting failure modes with *testNumberTooShortInConstructor* and *testNumberNonDigitsInConstructor*. Similarly, *TaskTest.java* verified the 20-character name and 50-character description bounds using *testExactBoundaryLengthsSuccess* (lines 48-61), and *AppointmentTest.java* used *futureDate* and *pastDate* variables in *@BeforeEach* (lines 28-34) to verify the 50-character description and ensure dates cannot be in the past (*testApptDateInPastCreation*). Across each of the three services, I confirmed that objects could be added, deleted, and updated by unique ID, while duplicate or non-existent IDs threw an *IllegalArgumentException*.

The overall quality and effectiveness of my JUnit tests across the three services were very high, shown by achieving 100% coverage in class, method, line, and branch metrics. Coverage percentage proves effectiveness here because distinct requirements were not hidden behind

compound conditionals. In *Contact.java*, *Task.java*, and *Appointment.java*, my validation helpers checked null fields (*if (field == null)*) and field lengths (*if (field.length() > 10)*) using separate *if* statements rather than lumping them together with logical OR operators. Separate test methods targeted each condition independently in both constructors and mutators, leaving no branch untested. I also covered critical edge cases in the service classes, such as post-condition verification in *testDeleteContactSuccess* (*ContactServiceTest.java* lines 53-63), where I confirmed that the deleted object returned *null* and then asserted that deleting the same ID a second time threw an *IllegalArgumentException*.

I ensured that my code was technically sound by following the correct order of procedures and strict assertions to enforce fail-fast validation and defensive programming practices. In *Task.java* (lines 17-27), all arguments are validated in private helpers (*validateTaskID*, *validateName*, *validateDescription*) before assigning instance variables, throwing an *IllegalArgumentException* immediately so partially initialized objects never exist in memory. Additionally, in *Appointment.java*, because *java.util.Date* is mutable, I implemented defensive copies in the constructor (*this.apptDate = new Date(apptDate.getTime())* on line 26), getter (line 36), and setter (line 46), and verified this encapsulation in *AppointmentTest.java* (lines 86-124) with *testDateImmutabilityViaConstructorDefensiveCopy*. On the testing side, I used *assertThrows* with lambda expressions in *TaskServiceTest.java* lines 37-45 (*testAddTaskDuplicateIdFails*) to isolate *service.addTask(duplicateTask)* and ensure the expected exception was thrown by the exact method under test.

I ensured that my code was efficient by optimizing performance with the in-memory data structure implementation, avoiding redundant boilerplate, and preventing shared mutable states between tests. In *ContactService.java*, *TaskService.java*, and *AppointmentService.java* (line 14),

objects are stored using a *HashMap* keyed by unique ID rather than an *ArrayList*, allowing insertions, deletions, duplicate checks (*containsKey*), and updates to execute in O(1) constant time rather than O(n) linear time. Similarly, in *Contact.java* (lines 122-129), instead of compiling a regular expression on every phone validation, I wrote a lightweight *isDigitOnly* loop using *Character.isDigit(str.charAt(i))*. Furthermore, in each test file, such as *TaskServiceTest.java* (lines 21-26), I used *@BeforeEach* (*setUp()*) to re-instantiate baseline objects before each test executes, avoiding repetitive construction and isolating tests from one another.

In developing the three services, I employed several software testing techniques to ensure quality code that meets client requirements. Most notably, I conducted unit testing with JUnit 5, testing isolated units of code such as constructors, getters, and setters independently using *@Test*, *@BeforeEach*, and assertion methods (*assertEquals*, *assertNull*, *assertThrows*). These tests also employed boundary value analysis by checking limits right at and just above maximum thresholds, such as testing exact 10, 20, 30, and 50-character strings in *ContactTest.java* (*testExactBoundaryLengthsSuccess*, lines 50-74) paired with 11, 21, 31, and 51-character strings in negative tests. This ties into negative and exception testing, where invalid inputs, such as *null* arguments, duplicate IDs, and past dates, confirm that the software failed safely by throwing an *IllegalArgumentException*. Finally, code coverage analysis determined that every conditional guard clause (such as *if (apptDate.before(new Date()))*) and exception branch was executed to ensure 100% coverage.

Despite thorough verification through unit testing, boundary analysis, negative testing, and coverage analysis, there are several testing techniques I did not employ. Because the services were developed and tested individually using in-memory hash maps, I did not conduct integration testing to combine modules or verify cross-functionality with a real database or API.

System testing would take this another step further, testing the complete application end-to-end from the database backend to the mobile user interface frontend, whereas my testing was conducted strictly through the IDE test runner. Furthermore, it is important to consider performance testing to evaluate responsiveness and stability under heavy concurrent load, as well as security testing to detect vulnerabilities like injection, broken authentication, or data leakage.

Each of these techniques has practical uses and implications depending on the software development project and situation. Techniques I employed like unit testing and boundary analysis are fast and inexpensive to automate, making them ideal during active Agile sprints and CI/CD pipelines where logic errors can be caught early. However, 100% unit test coverage in isolation does not ensure components will work together once connected. Thus, integration and system testing become critical in multi-tier mobile applications and microservice architectures where UIs, APIs, and databases interact. Similarly, while end-to-end, performance, and security testing cost more time and resources, they are essential before production releases in high-traffic or regulated domains to prevent outages and mitigate compliance, financial, and legal liabilities.

Assessing the mindset I adopted while working on this project, I employed a high degree of caution as a software tester by assuming any unguarded pathway or shared reference could be misused. Appreciating the complexity and interrelationships of the code was essential because separate components depend on one another's state integrity. For example, in this project, if an external caller mutated a shared *Date* object in memory, it could corrupt an appointment already stored inside the service layer, just as in a larger e-commerce system where modifying a shared customer address object could accidentally alter historical invoice records.

Limiting bias was an equally important part of my mindset. On the developer side, confirmation bias is a major concern when testing your own code because developers naturally gravitate toward proving their code works rather than trying to break it. For instance, a developer who writes a login or contact form might only test valid user entries because they subconsciously assume users will follow instructions, overlooking what happens when someone submits empty fields, emojis, or SQL injection strings. To limit my own bias, I treated the customer requirements as an independent checklist rather than relying on my memory of expected behavior, forcing myself to test every possible failure mode for each requirement.

Finally, maintaining strict discipline in our commitment to quality is vital as a software engineering professional. Cutting corners when writing or testing code, such as skipping negative tests to rush a feature before a sprint deadline or copy-pasting validation logic across classes, directly accumulates technical debt. In a real-world production environment, a single unvalidated update method or untested edge case can allow corrupted data into a live database, taking days of engineering time to trace and repair while eroding user trust. As a practitioner in the field, I plan to avoid technical debt by writing modular, reusable validation code, enforcing automated unit and regression tests in CI/CD pipelines before any code is merged, and conducting peer code reviews so fresh eyes can catch blind spots early.